AWS Web Application Architecture

System design, deployment topology, CloudFront caching, WAF, ALB, and the full request flow.

---

1. Architecture Overview

A typical AWS web application follows a three-tier architecture: an edge layer for global distribution and security, a compute layer inside a VPC for application logic, and a data layer for persistence and caching. Supporting services handle identity, observability, secrets, and CI/CD.

Layer	Services	Purpose
Edge	Route 53, CloudFront, WAF	DNS, CDN, caching, firewall
Compute	ALB, EC2 / ECS (multi-AZ)	Application logic, load balancing
Data	RDS, ElastiCache, S3	Database, cache, static assets
Supporting	IAM, CloudWatch, Secrets Manager, CodePipeline	Security, monitoring, deployments

2. Edge Layer

Route 53

Route 53 handles DNS resolution. A record (e.g. www.myapp.com) points to the CloudFront distribution domain (xxxxx.cloudfront.net). The preferred record type is an **Alias** record — works at the zone apex, no extra cost, and integrates natively with CloudFront.

CloudFront — Origins and Behaviors

CloudFront is configured with two key concepts:

- **Origins** — where CloudFront fetches content from (ALB for dynamic content, S3 for static assets).
- **Behaviors** — URL path rules that map requests to the correct origin.

Path pattern	Origin	Caching
/api/*	ALB	Disabled — dynamic content
/static/*	S3 bucket	Aggressive — long TTL
/* (default)	ALB	Per Cache-Control header

WAF — Web Application Firewall

WAF is attached directly to the CloudFront distribution. Every request is inspected before CloudFront forwards it to the origin, meaning malicious traffic is blocked at the edge globally — not inside your VPC.

WAF capability	Protects against
SQL injection / XSS rules	Common web exploits in request parameters
IP rate limiting	DDoS and brute-force attacks
Geo-blocking	Traffic from specific countries
Custom rules	Bad user agents, malformed headers, patterns

WAF can also be attached to ALB, API Gateway, or AppSync. Attaching at CloudFront is recommended for most web apps as it intercepts threats before they reach your AWS region.

3. Full Request Flow

#	Step	Detail
1	**User hits www.myapp.com**	Browser initiates DNS lookup
2	**Route 53**	Resolves domain to CloudFront distribution via Alias record
3	**WAF inspection**	Request inspected against WAF rules; blocked if malicious
4	**CloudFront behavior match**	URL path matched to behavior rule (e.g. /api/* goes to ALB)
5	**Cache check**	Cache HIT: response returned from edge. Cache MISS: forward to origin
6	**ALB**	Receives request, routes to healthy EC2/ECS target in target group
7	**EC2 / ECS**	Application processes request and returns response
8	**CloudFront caches response**	Response cached at edge per Cache-Control headers for next request

4. Securing the ALB from Direct Access

Since CloudFront has a public DNS, someone could bypass WAF by hitting the ALB URL directly. Two approaches prevent this:

Option A — Custom origin header

Add a secret header in CloudFront (e.g. X-Origin-Secret: abc123) and configure the ALB listener rule to only forward requests containing that header. Direct requests without the header are rejected.

Option B — CloudFront managed prefix list

AWS publishes a managed prefix list of all CloudFront edge IP ranges. Add this as the only allowed inbound source in the ALB security group. Requests from any other IP are dropped at the network level.

5. CloudFront Caching and TTL

There are two ways to control caching — from your origin application via Cache-Control headers, or directly in CloudFront via a Cache Policy.

Option 1 — Cache-Control headers from your origin (recommended)

Set headers in your Spring Boot controller and CloudFront respects them:

```
@GetMapping("/products")
public ResponseEntity<List<Product>> getProducts() {
return ResponseEntity.ok()
.header("Cache-Control", "public, max-age=300") // 5 minutes
.body(productService.getAll());
}
```

Cache-Control value	TTL	Use case
public, max-age=300	5 min	Product listings, semi-static data
public, max-age=3600	1 hour	Rarely changing data
public, max-age=86400	24 hours	Static reference data
public, s-maxage=600	10 min	CDN-specific TTL (overrides max-age for CloudFront only)
no-cache	0	Always revalidate with origin
no-store	0	Never cache — sensitive data
private	0	User-specific data — CloudFront skips

Tip: s-maxage is useful when you want CloudFront to cache longer than the browser TTL.

Option 2 — CloudFront Cache Policy

Set TTL directly in CloudFront behavior, independent of your app headers:

```
resource "aws_cloudfront_cache_policy" "api_cache" {
min_ttl = 0
default_ttl = 300 # used when origin sends no Cache-Control
max_ttl = 3600 # CloudFront never caches longer than this
}
```

TTL resolution order:

Condition	TTL used
Origin sends Cache-Control max-age	That value, clamped between min_ttl and max_ttl
Origin sends no Cache-Control	default_ttl from cache policy
Always	Final TTL never goes below min_ttl or above max_ttl

Per-behavior TTL (recommended pattern):

Behavior / path	TTL setting	Reason
/static/*	max_ttl = 86400 (24h)	Images, CSS, JS — rarely change
/api/products	max_ttl = 300 (5 min)	Product data — changes occasionally
/api/user/*	no-store	User-specific — never cache
/api/orders/*	no-store	Transactional — never cache

Cache invalidation

When data changes, invalidate the CloudFront cache via AWS CLI or from your Spring Boot app:

```
aws cloudfront create-invalidation \
--distribution-id ABCDEF123456 \
--paths "/api/products"
```

6. Caching with Different Query Parameters

By default CloudFront ignores query parameters and returns the same cached response regardless of them. You must explicitly tell CloudFront to include query strings in the cache key.

The problem without query string config:

```
GET /api/products?category=electronics → cache key: /api/products
GET /api/products?category=furniture → cache key: /api/products (SAME — WRONG!)
```

Fix — include query strings in cache key (Terraform):

```
query_strings_config {
query_string_behavior = "whitelist"
query_strings {
items = ["category", "page", "sort", "limit"]
}
```

```
}
```

Query string behavior options:

Behavior	Cache key	Use when
none	Path only — params ignored	Same response for all param combinations
all	Full URL including all params	Every param changes the response
whitelist	Path + only listed params	Only specific params affect the response

Real-world cache behaviour with whitelist ["category", "page"]:

Request	Result
/api/products?category=electronics	MISS — hits backend, response cached
/api/products?category=electronics (again)	HIT — served from CloudFront cache
/api/products?category=furniture	MISS — separate cache entry
/api/products?category=electronics&page;=2	MISS — separate cache entry (page differs)
/api/products?category=electronics&utm;=google	HIT — utm ignored, same cache as row 1
/api/orders?userId=123	MISS — no-store header, never cached

Recommended approach: use whitelist and only include params that actually change your response. This prevents cache pollution from tracking params (utm_source, fbclid, etc.) and gives fine-grained control per CloudFront behavior.

7. Data Layer

Service	Role	Key config
RDS (Multi-AZ)	Primary relational database	Automatic failover to standby in another AZ
ElastiCache (Redis)	In-memory cache / session	Reduces DB load; stores session tokens
S3	Static assets and backups	Served via CloudFront; versioning for backups

8. Supporting Services

Service	Purpose
IAM	Access control — roles, policies, least-privilege permissions for all AWS resources

CloudWatch	Centralized logging, metrics, dashboards, and alarms across all services
Secrets Manager	Secure storage and auto-rotation of DB credentials, API keys, and certificates
CodePipeline	CI/CD pipeline — source, build (CodeBuild), and deploy (CodeDeploy / ECS) stages